Propuesta de Proyecto:

Sistema inteligente distribuido de monitorización de calidad del aire (CO_2)

Integrantes:

Jara Blasco Arroyo

David Henao Hernández

Miguel Ángel Osuna Galindo

1. Objetivos del Proyecto

El proyecto tiene como objetivo diseñar un sistema de monitorización ambiental inteligente que resuelva la falta de visibilidad sobre la calidad del aire en espacios compartidos:

- **Monitorización Precisa y Multivariable:** Medir niveles de eCO_2, compuestos orgánicos volátiles (TVOC), temperatura y humedad.
- **Interfaz de Usuario de Doble Nivel:** Proporcionar información detallada mediante una **pantalla LCD** (datos técnicos) y alertas rápidas mediante **LEDs de colores** (estado semafórico).
- **Arquitectura IoT Distribuida:** Separar el punto de medición (nodo sensor) del punto de visualización (nodo central) para mejorar la representatividad de los datos.
- **Promoción de Espacios Saludables:** Facilitar la toma de decisiones para la ventilación mecánica o manual en entornos críticos como aulas u oficinas.

2. Arquitectura de Hardware

El sistema utiliza una configuración maestro-esclavo mediante dos microcontroladores de última generación:

- **Nodo de Adquisición (ESP32-v4)**

Es el dispositivo encargado de la lectura física del entorno. Su tamaño compacto permite colocarlo en lugares estratégicos.

Sensor SGP30: Encargado de la medición de **eCO_2** y **TVOC**.

Sensor Si7021: Encargado de la lectura de **temperatura y humedad**.

Sensor de Distancia: Implementado en este nodo para activar la pantalla o las alertas solo cuando detecte presencia cercana, optimizando el consumo.

Transmisión: Envía los paquetes de datos de forma inalámbrica al nodo central.

- **Nodo de Visualización y Control (ESP32-C3)**

Actúa como la unidad central de proceso y la interfaz principal para el usuario.

Pantalla LCD (I2C): Muestra de forma clara y textual los valores en tiempo real (ej: "CO2: 740 ppm").

Sistema de LEDs (Semaforización):

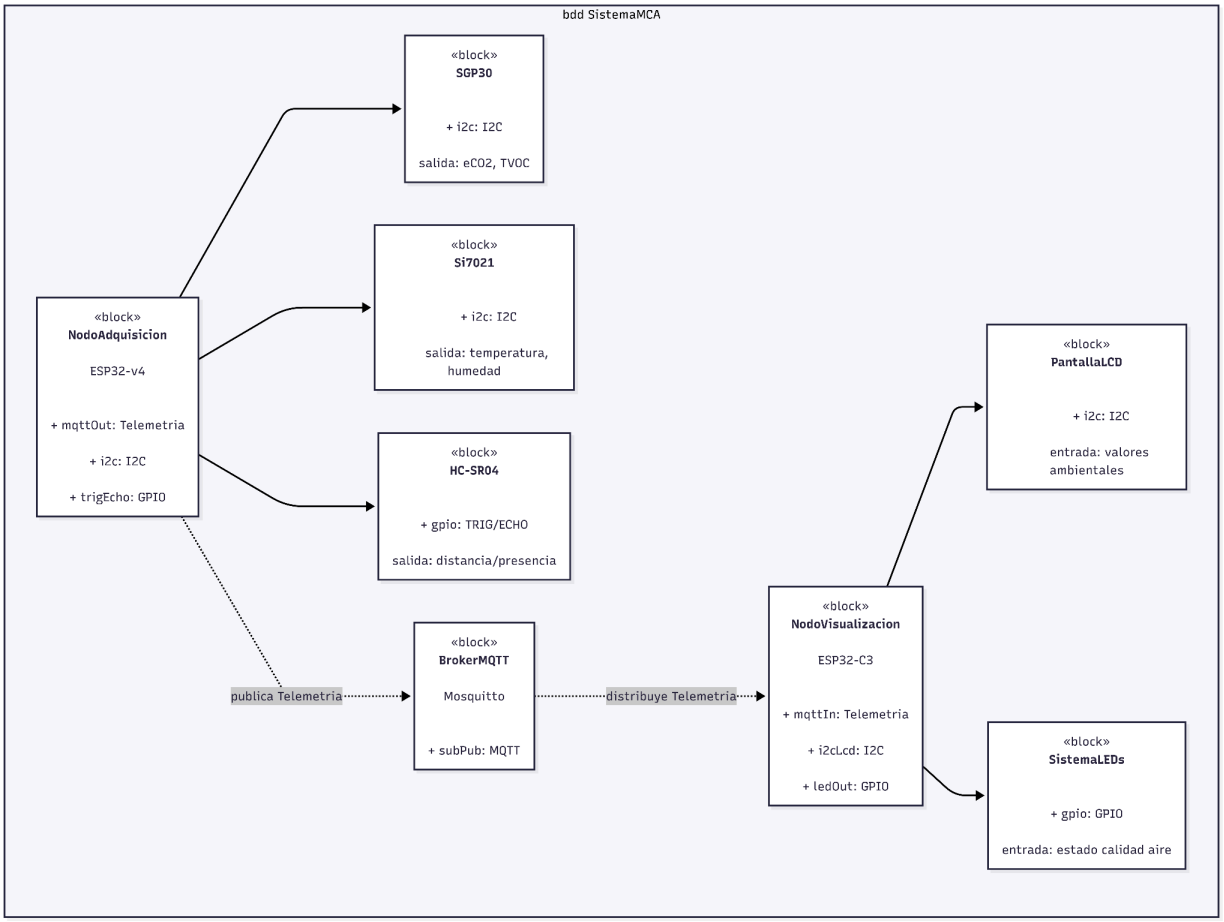
Verde: Calidad óptima.

Amarillo: Calidad moderada (aviso de ventilación).

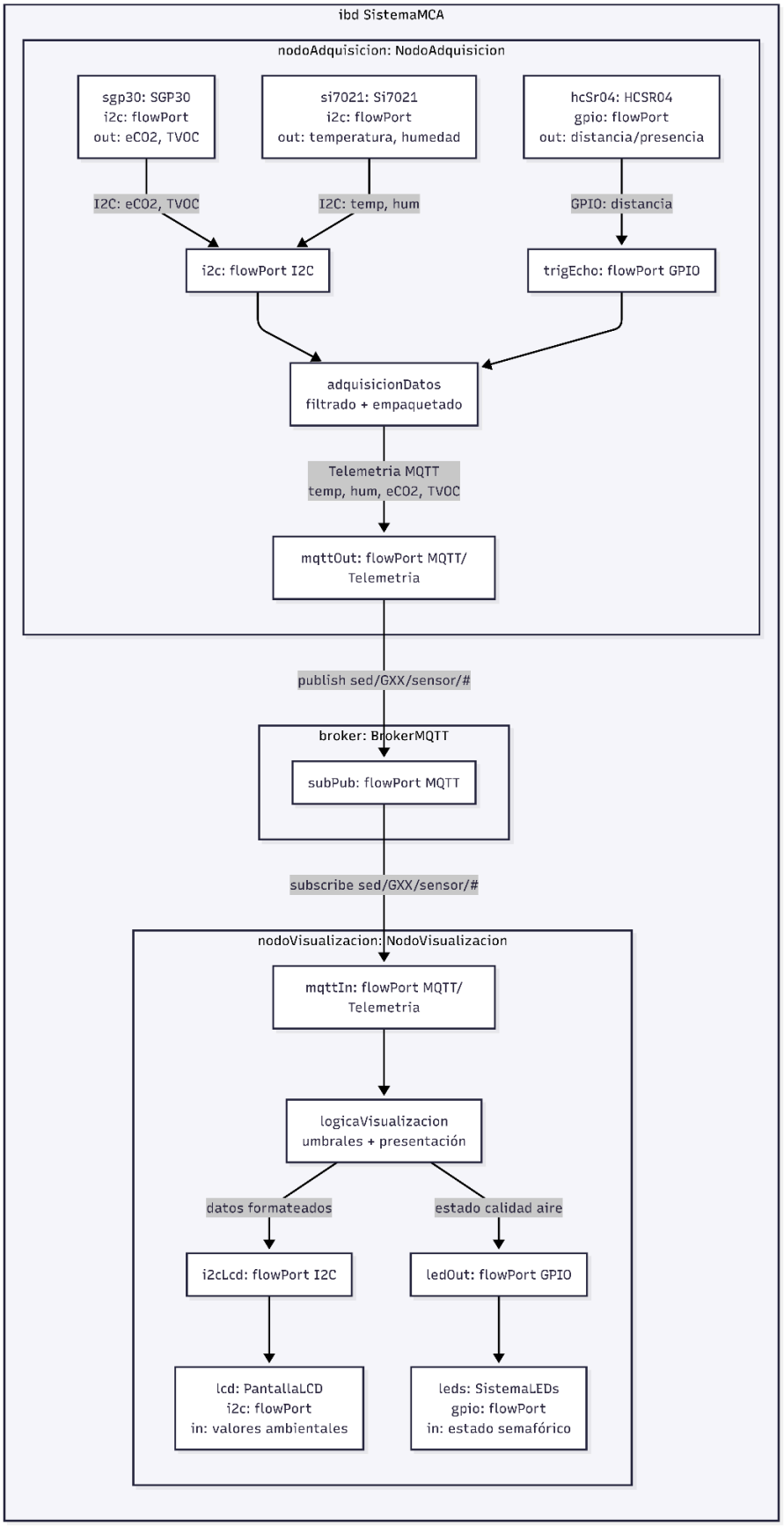
Rojo: Calidad crítica (acción inmediata).

3. Diagrama de bloques

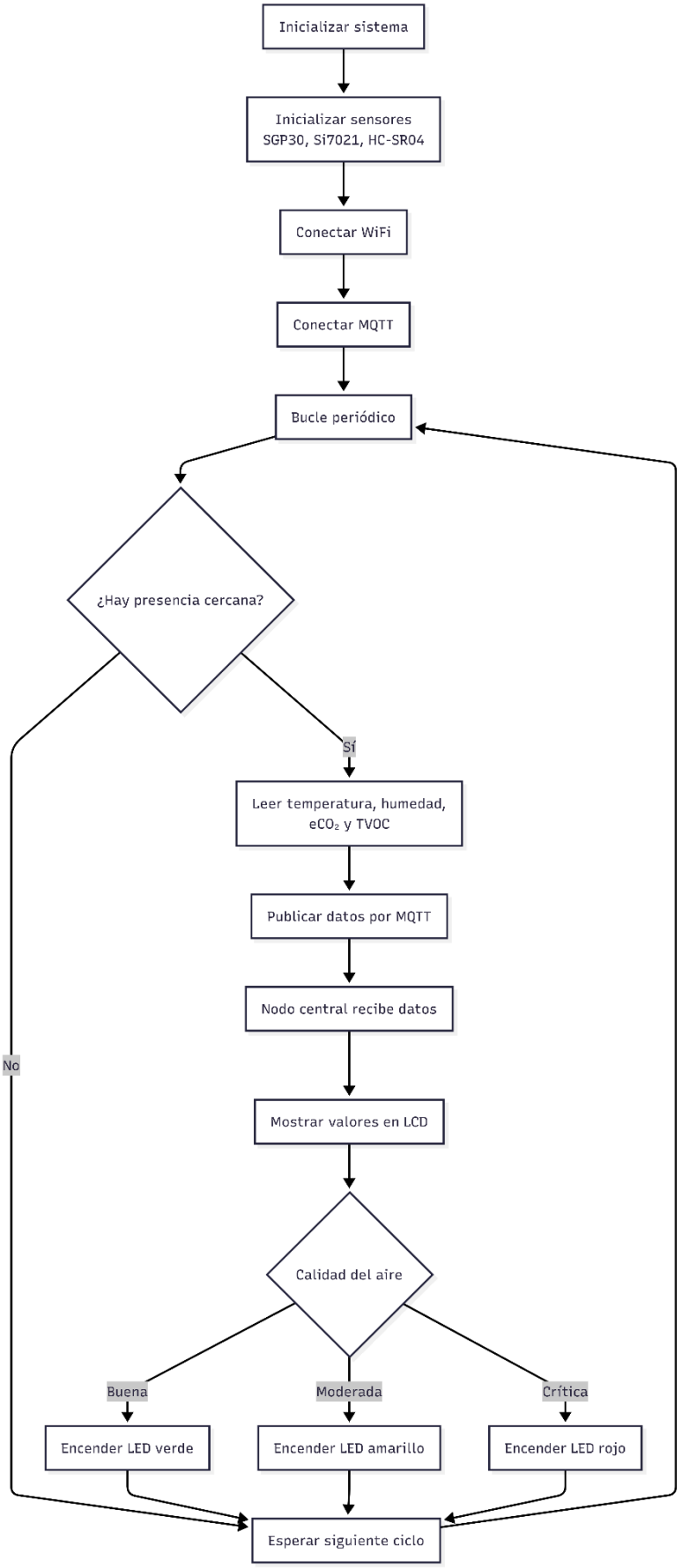
BDD:



IBD:

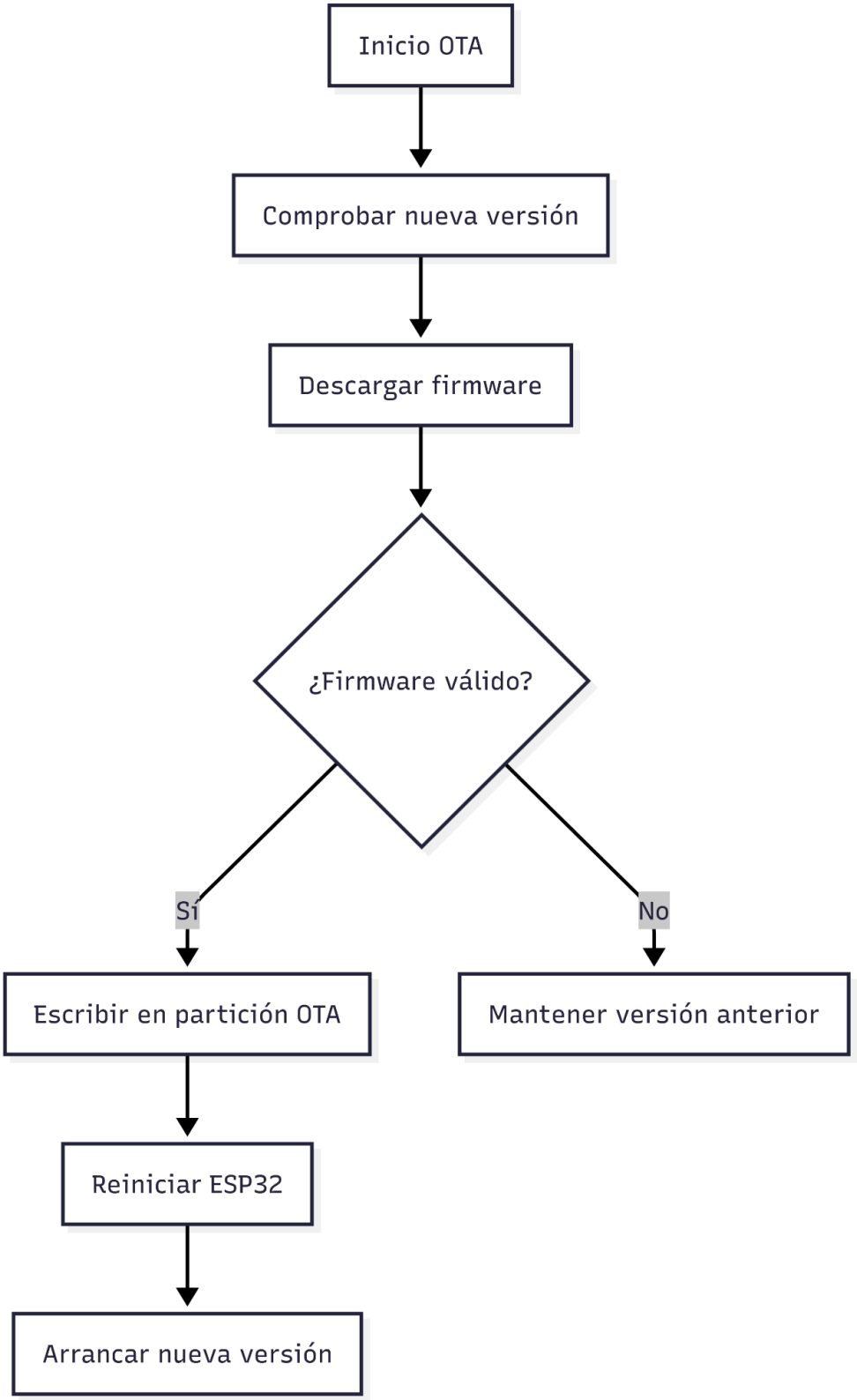

ibd SistemaMCA
nodoAdquisicion: NodoAdquisicion
sgp30: SGP30
i2c: flowPort
out: eCO2, TVOC
si7021: Si7021
i2c: flowPort
out: temperatura, humedad
hcSr04: HCSR04
gpio: flowPort
out: distancia/presencia
I2C: eCO2, TVOC
I2C: temp, hum
GPIO: distancia
i2c: flowPort I2C
trigEcho: flowPort GPIO
adquisicionDatos
filtrado + empaquetado
Telemetria MQTT
temp, hum, eCO2, TVOC
mqttOut: flowPort MQTT/
Telemetria
publish sed/GXX/sensor/#
broker: BrokerMQTT
subPub: flowPort MQTT
subscribe sed/GXX/sensor/#
nodoVisualizacion: NodoVisualizacion
mqttIn: flowPort MQTT/
Telemetria
logicaVisualizacion
umbrales + presentación
datos formateados
estado calidad aire
i2cLcd: flowPort I2C
ledOut: flowPort GPIO
lcd: PantallaLCD
i2c: flowPort
in: valores ambientales
leds: SistemaLEDs
gpio: flowPort
in: estado semafórico

Flujo de comportamiento:



4. Sistema OTA

Para subir otras versiones de manera remota, se ha hecho al igual que en la práctica, siguiendo el siguiente proceso:



Análisis de resiliencia del sistema

El sistema IoT desarrollado presenta varios mecanismos de resiliencia orientados a mantener la operación incluso ante fallos de conectividad, errores de sensores o reinicios inesperados. La arquitectura está compuesta por dos nodos:

- Nodo de adquisición y publicación de sensores (ESP32 v4)
- Nodo de visualización y alerta mediante LEDs (ESP32 C3)

Ambos utilizan WiFi, MQTT y multitarea con FreeRTOS.

1. Gestión de pérdida de conexión Wi-Fi

El sistema detecta automáticamente la pérdida de conexión mediante el evento:

- WIFI_EVENT_STA_DISCONNECTED

Cuando ocurre una desconexión:

- wifi_connected = false;
- mqtt_connected = false;

Además, se ejecuta una tarea automática de reconexión:

- wifi_reconnect_task()

la cual intenta recuperar la conexión mediante:

- esp_wifi_connect();

Configuración utilizada:

- Nodo sensores: WIFI_RECONNECT_MS = 5000 ms

- Nodo visualización: WIFI_RECONNECT_MS = 10000 ms

Esto permite tolerancia ante pérdidas temporales de red, reinicios del router o caídas de señal.

2. Gestión de fallos MQTT

La conexión MQTT se supervisa de manera independiente a la conexión Wi-Fi.

Cuando el broker MQTT deja de estar disponible:

- MQTT_EVENT_DISCONNECTED

el sistema marca MQTT como desconectado:

- mqtt_connected = false;

y ejecuta reconexiones automáticas mediante:

- esp_mqtt_client_reconnect(mqtt_client);

Configuración utilizada:

- Nodo sensores: MQTT_RECONNECT_MS = 5000 ms

- Nodo visualización: MQTT_RECONNECT_MS = 15000 ms

También se implementó Last Will:

```
.session.last_will = {
 .topic = TOPIC_STATUS,
 .msg = "Offline",
 .qos = 1,
 .retain = 1,
},
```

Si un ESP32 falla o se apaga abruptamente, el broker publica automáticamente el estado “Offline”.

3. Gestión de errores de sensores

Cada sensor se inicializa de forma independiente:

- init_si7021();
- init_sgp30();
- ultrasonic_init();

Si un sensor falla:

- se deshabilita,

- pero el sistema continúa funcionando.

Variables utilizadas:

- si7021_available

- sgp30_available

- ultrasonic_available

Antes de utilizar los datos, el sistema verifica:

- if (ret == ESP_OK)

Además, algunos valores se limitan manualmente para evitar lecturas inválidas:

- if (*humidity > 100.0f) *humidity = 100.0f;
- if (*humidity < 0.0f) *humidity = 0.0f;

4. Gestión de reinicios y OTA

El sistema incorpora actualizaciones OTA mediante Mender.

Cuando se instala un nuevo firmware:

- esp_restart();

el ESP32 reinicia automáticamente.

Tras iniciar correctamente, el firmware se valida mediante:

- mender_flash_confirm_image();

Esto permite rollback automático si la actualización falla.

Además, el sistema verifica errores de memoria NVS:

- ESP_ERR_NVS_NO_FREE_PAGES
- ESP_ERR_NVS_NEW_VERSION_FOUND

y realiza recuperación automática mediante:

- nvs_flash_erase();

5. Resiliencia del nodo visualizador

Cuando MQTT se desconecta, el nodo apaga automáticamente todos los LEDs:

- gpio_set_level(LED_VERDE, 0);
- gpio_set_level(LED_AMARILLO, 0);
- gpio_set_level(LED_ROJO, 0);

Esto evita mostrar información desactualizada.

El sistema también conserva las últimas mediciones recibidas:

- last_temp

- last_hum

- last_eco2

- last_tvoc

permitiendo mantener parcialmente el estado ante pérdidas temporales de datos.

Conclusión

El sistema presenta una arquitectura resiliente y robusta para un entorno IoT embebido. Implementa recuperación automática ante fallos de Wi-Fi y MQTT, tolerancia a errores de sensores, funcionamiento degradado y actualizaciones OTA seguras.

Las principales características de resiliencia son:

- Reconexión automática Wi-Fi y MQTT
- Gestión independiente de sensores
- Detección de desconexiones
- Recuperación automática tras fallos
- OTA segura con rollback
- Arquitectura distribuida desacoplada
- Prevención de publicación inválida
- lecturas corruptas
- errores silenciosos